

JSST 2026

An Integrated CFD Workflow Platform for In-House Solvers

Modular Architecture and AI-Assisted Implementation

Hsueh-Jui Lu, Sheng-Jer Chen, Fang-An Kuo

National Center for High-performance Computing

National Institutes of Applied Research, Taiwan

MOTIVATION

In-house solvers are numerically mature — and rarely adopted

The barrier is the workflow, not the numerics.

- One simulation means coordinating **five separate tools**
- Manual file conversions and inter-tool data transfers at every stage
- Solver-specific configuration, with little or no GUI support
- Even for a well-validated solver, familiarization takes **weeks**

UNICONES

A good solver with a familiarization cost measured in weeks.

- Compressible Euler / Navier–Stokes solver based on the **CESE** method
- Developed at NCHC — National Institutes of Applied Research, Taiwan
- Openly available academically, yet still requires weeks of guided effort
- High numerical maturity, limited adoption — *exactly* the gap this work targets

This paper contributes three things

i

A modular solver-adapter interface

Lightweight integration of heterogeneous solvers
— lowering the adoption barrier for new users.

ii

A human–AI development methodology

Documented, with empirical observations on how
the contribution was actually distributed.

iii

A generalizable design strategy

Workflow integration for research groups
maintaining in-house solvers with limited
software-engineering resources.

One environment, the whole pipeline

Four modules, a single PyQt6 GUI, and automatic hand-off between every stage.

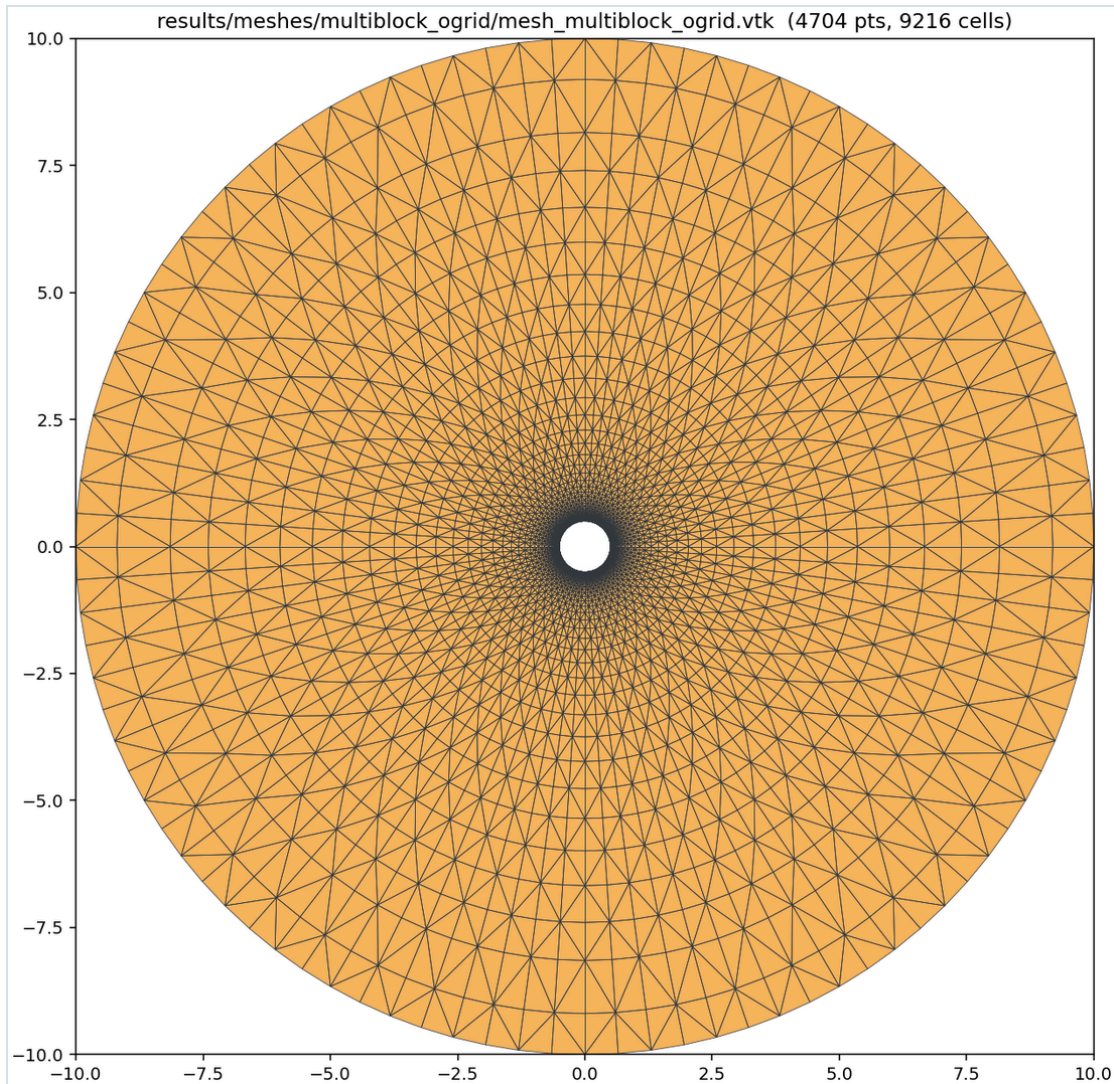


Fig. 1 Data flow of the integrated CFD workflow platform.

PreProcessor — geometry preparation

- Interactive geometry canvas
- Multiple point-spacing strategies
- Automatic sharp-corner detection
- Per-segment resampling
- Standalone C++ backend

Mesh Generator — 2D hybrid meshes



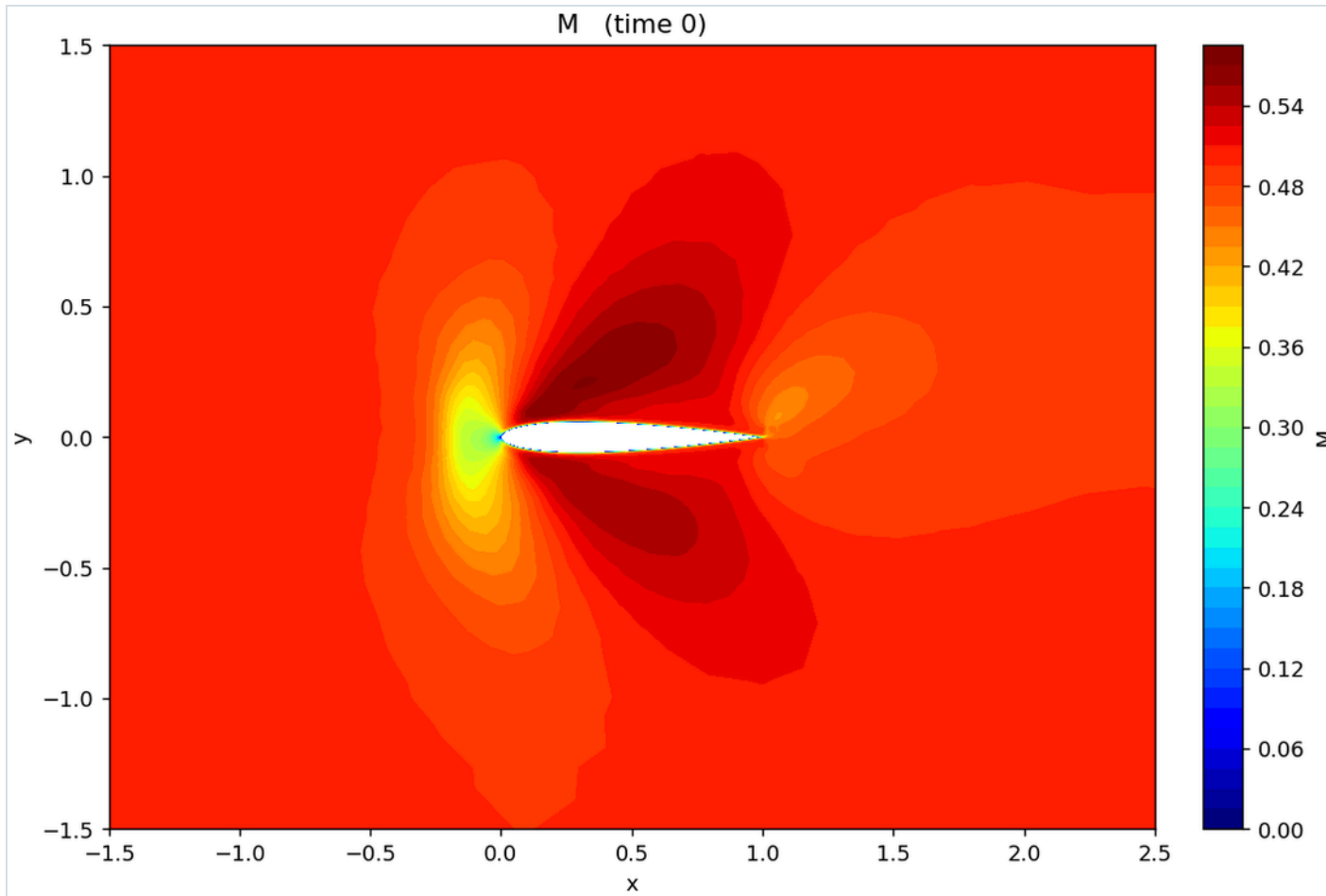
Structured boundary-layer quads growing from the body, unstructured triangles filling the far field.

- Boundary-layer quads by **node advancement**
- Far-field triangles via Gmsh **Frontal-Delaunay**
- ~1,500 lines of C++
- Exports VTK and STAR-CD (.vrt/.cel/.bnd)
- Inter-stage conversions handled automatically

Core Solver — execution and monitoring

- Input format conversion
- Optional MPI domain decomposition
- Executes solvers distributed as shared libraries, via a built-in DLL call template
- A background worker parses stdout in real time for **live convergence monitoring**

Results Visualization



Cell-centred Mach number on a NACA 0012, rendered on the embedded canvas.

- Parses Tecplot-format output
- Cell-centred scalar fields on an embedded matplotlib canvas
- Contours, streamlines and vector glyphs
- Interactive colormap and field-variable controls

The Solver Adapter

Each adapter implements **three interface methods**. Everything upstream stays unchanged.

3

interface methods

<150

lines of Python per new solver

0

changes to upstream modules

Three methods, one interface

Method	Input	Output	Purpose
<code>convert_mesh(src, dst)</code>	VTK mesh path	Solver-format mesh file(s)	Converts the platform mesh to the solver's input format — e.g. STAR-CD .vrt/.cel/.bnd for UNICONES.
<code>parse_stdout(line) → dict</code>	One line of solver stdout	Convergence quantities, or None	Extracts iteration, CFL and residual for live monitoring; returns None for non-data lines.
<code>read_results(path) → dataset</code>	Solver result file path	Structured field dataset	Parses solver-specific output for the visualization module, fully decoupling it from the output format.

Table 1 Solver Adapter interface definition.

How it was built

- LLM coding assistants in **agentic mode** — reading, writing and executing project files, not merely generating text
- Experts defined the pipeline architecture, the numerical algorithms, the format-compatibility chain, and physical validation
- AI generated GUI boilerplate, file parsers, and C++ routines from algorithmic descriptions
- Each feature converged in **2–5 rounds** of describe → generate → execute → validate → correct

Text was not enough

- The expert described outward node advancement in natural language; the AI produced a first C++ implementation
- Text description alone **could not resolve the geometric failures**
- Actual runtime output was needed to find self-intersections at sharp trailing edges and wrong fan-node counts at convex corners
- More precise prompts consistently reduced the number of correction rounds
- Even a fix the AI reported as correct still needed **physical validation**, to catch discrepancies that appeared only in real use

What the logs say

Reported as case-study observations, not controlled experimental results.

~70–80%

of code AI-generated, by line count

2–5

median correction rounds per feature

~4–6 weeks

total effort, vs. 3–6 months conventional

1

domain expert — no dedicated GUI engineer

The contribution was unevenly distributed

GUI boilerplate, widget layouts, signal-slot wiring, file I/O

~80–90%



Cross-component integration and domain-specific numerical logic

~40–60%



The 69-file, 18,000-line PyQt6 GUI was highly AI-productive for boilerplate — but workflow-level decisions (mode switching, cross-panel consistency) needed explicit human specification. The IBM and non-IBM output modes were discovered, only in live testing, to produce different stdout structures.

EVALUATION

Before and after

Eliminating manual format conversion and inter-tool transfer at every stage.

5 → 1

separate tools required

~20 → ~5

manual steps per simulation

**~2 weeks → ~2
days**

time for a new user to become familiar

Indicative figures from informal observation, not controlled experimental results.

DISCUSSION

The oversight that proved indispensable

Three layers, and the method fails without all three.

1

Precise problem description

More careful phrasing consistently reduced correction rounds.

2

Execution-based diagnosis

Runtime output found the edge cases that description could not.

3

Physical validation

An expert still had to catch discrepancies visible only in real use.

Without domain expertise at each layer, AI-assisted scientific software development risks producing **plausible but incorrect** implementations.

What we take away

- A unified CFD workflow platform for in-house solvers, demonstrated with UNICONES — without modifying the solver core
- Tools reduced **5 → 1**, workflow steps **~20 → ~5**, onboarding **~2 weeks → ~2 days**
- The Solver Adapter — **under 150 lines of Python per new solver** — decouples solver-specific logic from the platform core
- A generalizable strategy for groups maintaining in-house solvers with limited software-engineering resources

FUTURE WORK

Where this goes next

- Extend the platform to **3D mesh generation**
- A formal user study with standardized usability metrics
- Web-based deployment, to further lower the barrier for CFD education and research

Thank you — questions welcome.

This work was made possible by the academic resources and advanced research infrastructure of the National Center for High-performance Computing, National Institutes of Applied Research, Taiwan, with partial funding from the fundamental R&D program of NSTC, Taiwan.